

13 缓存与效果的极限拉扯：从MHA、MQA、GQA到MLA

May By 苏剑林 | 2024-05-13 | 187469位读者

前几天，幻方发布的DeepSeek-V2引起了大家的热烈讨论。首先，最让人哗然的是1块钱100万token的价格，普遍比现有的各种竞品API便宜了两个数量级，以至于有人调侃“这个价格哪怕它输出乱码，我也会认为这个乱码是一种艺术”；其次，从模型的技术报告看，如此便宜的价格背后的关键技术之一是它新提出的MLA (Multi-head Latent Attention)，这是对GQA的改进，据说能比GQA更省更好，也引起了读者的广泛关注。

接下来，本文将跟大家一起梳理一下从MHA、MQA、GQA到MLA的演变历程，并着重介绍一下MLA的设计思路。

MHA

MHA (Multi-Head Attention)，也就是多头注意力，是开山之作《Attention is all you need》所提出的一种Attention形式，可以说它是当前主流LLM的基础工作。在数学上，多头注意力MHA等价于多个独立的单头注意力的拼接，假设输入的（行）向量序列为 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_l$ ，其中 $\mathbf{x}_i \in \mathbb{R}^d$ ，那么MHA可以形式地记为

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})} \quad (1)$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_q^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{k}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_k^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_k^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{v}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(s)} &\in \mathbb{R}^{d \times d_v} \end{aligned}$$

简单起见，这里省略了Attention矩阵的缩放因子。实践上，常见的设置是 $d_k = d_v = d/h$ ，对于LLAMA2-7b有 $d = 4096, h = 32, d_k = d_v = 128$ ，LLAMA2-70b则是 $d = 8192, h = 64, d_k = d_v = 128$

由于这里只考虑了主流的自回归LLM所用的Causal Attention，因此在token by token递归生成时，新预测出来的第 $t+1$ 个token，并不会影响到已经算好的 $\mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}$ ，因此这部分结果我们可以缓存下来供后续生成调用，避免不必要的重复计算，这就是所谓的KV Cache。

而后面的MQA、GQA、MLA，都是围绕“如何减少KV Cache同时尽可能地保证效果”这个主题发展而来的产物。

瓶颈

一个自然的问题是：为什么降低KV Cache的大小如此重要？

众所周知，一般情况下LLM的推理都是在GPU上进行，单张GPU的显存是有限的，一部分我们要用来存放模型的参数和前向计算的激活值，这部分依赖于模型的体量，选定模型后它就是个常数；另外一部分我们要用来存放模型的KV Cache，这部分不仅依赖于模型的体量，还依赖于模型的输入长度，也就是在推理过程中是动态增

长的，当Context长度足够长时，它的大小就会占主导地位，可能超出一张卡甚至一台机（8张卡）的总显存量。

在GPU上部署模型的原则是：能一张卡部署的，就不要跨多张卡；能一台机部署的，就不要跨多台机。这是因为“卡内通信带宽 > 卡间通信带宽 > 机间通信带宽”，由于“木桶效应”，模型部署时跨的设备越多，受设备间通信带宽的“拖累”就越大，事实上即便是单卡H100内SRAM与HBM的带宽已经达到了3TB/s，但对于Short Context来说这个速度依然还是推理的瓶颈，更不用说更慢的卡间、机间通信了。

所以，减少KV Cache的目的就是要实现在更少的设备上推理更长的Context，或者在相同的Context长度下让推理的batch size更大，从而实现更快的推理速度或者更大的吞吐总量。当然，最终目的都是为了实现更低的推理成本。

要想更详细地了解这个问题，读者可以进一步阅读《FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness》、《A guide to LLM inference and performance》、《LLM inference speed of light》等文章，这里就不继续展开了（主要是笔者水平也有限，唯恐说多错多）。

MQA

MQA，即“Multi-Query Attention”，是减少KV Cache的一次非常朴素的尝试，首次提出自《Fast Transformer Decoding: One Write-Head is All You Need》，这已经是2019年的论文了，这也意味着早在LLM火热之前，减少KV Cache就已经是研究人员非常关注的一个课题了。

MQA的思路很简单，直接让所有Attention Head共享同一个K、V，用公式来说，就是取消MHA所有的 k, v 的上标 (s) ：

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})} \quad (2)$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_q^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{k}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_k^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_k^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{v}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(s)} &\in \mathbb{R}^{d \times d_v} \end{aligned}$$

使用MQA的模型包括PaLM、StarCoder、Gemini等。很明显，MQA直接将KV Cache减少到了原来的 $1/h$ ，这是非常可观的，单从节省显存角度看已经是天花板了。

效果方面，目前看来大部分任务的损失都比较有限，且MQA的支持者相信这部分损失可以通过进一步训练来弥补。此外，注意到MQA由于共享了K、V，将会导致Attention的参数数量减少了将近一半，而为了模型总参数量的不变，通常会相应地增大FFN/GLU的规模，这也能弥补一部分效果损失。

GQA

然而，也有人担心MQA对KV Cache的压缩太严重，以至于会影响模型的学习效率以及最终效果。为此，一个MHA与MQA之间的过渡版本GQA（Grouped-Query Attention）应运而生，出自论文《GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints》，是去年的工作。

事后看来，GQA的思想也很朴素，它就是将所有Head分为 g 个组（ g 可以整除 h ），每组共享同一对K、V，用数学公式表示为

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention} \left(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(\lceil sg/h \rceil)}, \mathbf{v}_{\leq t}^{(\lceil sg/h \rceil)} \right) \triangleq \frac{\sum_{i \leq t} \exp \left(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(\lceil sg/h \rceil)\top} \right) \mathbf{v}_i^{(\lceil sg/h \rceil)}}{\sum_{i \leq t} \exp \left(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(\lceil sg/h \rceil)\top} \right)} \quad (3)$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_q^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{k}_i^{(\lceil sg/h \rceil)} &= \mathbf{x}_i \mathbf{W}_k^{(\lceil sg/h \rceil)} \in \mathbb{R}^{d_k}, & \mathbf{W}_k^{(\lceil sg/h \rceil)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{v}_i^{(\lceil sg/h \rceil)} &= \mathbf{x}_i \mathbf{W}_v^{(\lceil sg/h \rceil)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(\lceil sg/h \rceil)} &\in \mathbb{R}^{d \times d_v} \end{aligned}$$

这里的 $\lceil \cdot \rceil$ 是上取整符号。GQA提供了MHA到MQA的自然过渡，当 $g = h$ 时就是MHA， $g = 1$ 时就是MQA，当 $1 < g < h$ 时，它只将KV Cache压缩到 g/h ，压缩率不如MQA，但同时也提供了更大的自由度，效果上更有保证。GQA最知名的使用者，大概是Meta开源的LLAMA2-70B，以及LLAMA3全系列，此外使用GQA的模型还有TigerBot、DeepSeek-V1、StarCoder2、Yi、ChatGLM2、ChatGLM3等，相比使用MQA的模型更多（ChatGLM虽然在它的介绍中说自己是MQA，但实际是 $g = 2$ 的GQA）。

在llama2/3-70B中，GQA的 $g = 8$ ，其他用了GQA的同体量模型基本上也保持了这个设置，这并非偶然，而是同样出于推理效率的考虑。我们知道，70B这个体量的模型，如果不进行极端的量化，那么不可能部署到单卡（A100/H100 80G）上。单卡不行，那么就能单机了，一般情况下一台机可以装8张卡，刚才我们说了，Attention的每个Head实际上是独立运算然后拼接起来的，当 $g = 8$ 时，正好可以每张卡负责计算一组K、V对应的Attention Head，这样可以在尽可能保证K、V多样性的同时最大程度上减少卡间通信。

MLA

有了MHA、MQA、GQA的铺垫，我们理解MLA（Multi-head Latent Attention）就相对容易一些了。DeepSeek-V2的技术报告里是从低秩投影的角度引入MLA的，以至于有部分读者提出“为什么LoRA提出这么久了，直到MLA才提出对KV Cache低秩分解的做法”之类的疑问。

然而，笔者认为低秩投影这个角度并不贴近本质，因为要说低秩投影的话，事实上只要我们将GQA的所有K、V叠在一起，就会发现GQA也相当于在做低秩投影：

$$\underbrace{[\mathbf{k}_i^{(1)}, \dots, \mathbf{k}_i^{(g)}, \mathbf{v}_i^{(1)}, \dots, \mathbf{v}_i^{(g)}]}_{\mathbf{c}_i \in \mathbb{R}^{g(d_k + d_v)}} = \mathbf{x}_i \underbrace{[\mathbf{W}_k^{(1)}, \dots, \mathbf{W}_k^{(g)}, \mathbf{W}_v^{(1)}, \dots, \mathbf{W}_v^{(g)}]}_{\mathbf{W}_c \in \mathbb{R}^{d \times g(d_k + d_v)}} \quad (4)$$

这里我们将所有 $\mathbf{k}_i^{(s)}, \mathbf{v}_i^{(s)}$ 拼在一起记为 $\mathbf{c}_i$ ，相应的投影矩阵也拼在一起记为 $\mathbf{W}_c$ ，注意到一般都有 $d_c = g(d_k + d_v) < d$ ，所以 $\mathbf{x}_i$ 到 $\mathbf{c}_i$ 的变换就是一个低秩投影。所以，MLA的本质改进不是低秩投影，而是低秩投影之后的工作。

Part 1

GQA在投影之后做了什么呢？首先它将向量对半分两份分别作为K、V，然后每一份又均分为 g 份，每一份复制 h/g 次，以此来“凑”够 h 个Attention Head所需要的K、V。我们知道分割、复制都是简单的线性变换，所以MLA的第一个想法是将这些简单的线性变换换成一般的线性变换，以增强模型的能力：

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})} \quad (5)$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_q^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{k}_i^{(s)} &= \mathbf{c}_i \mathbf{W}_k^{(s)} \in \mathbb{R}^{d_k}, & \mathbf{W}_k^{(s)} &\in \mathbb{R}^{d_c \times d_k} \\ \mathbf{v}_i^{(s)} &= \mathbf{c}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(s)} &\in \mathbb{R}^{d_c \times d_v} \\ \mathbf{c}_i &= \mathbf{x}_i \mathbf{W}_c \in \mathbb{R}^{d_c}, & \mathbf{W}_c &\in \mathbb{R}^{d \times d_c} \end{aligned}$$

然而，理论上这样是能增加模型能力，但别忘了GQA的主要目的是减少KV Cache，出于节省计算和通信成本的考虑，我们一般会缓存的是投影后的 $\mathbf{k}_i, \mathbf{v}_i$ 而不是投影前的 $\mathbf{c}_i$ 或 $\mathbf{x}_i$ ，而MLA的这个做法，通过不同的投影矩阵再次让所有的K、V Head都变得各不相同，那么KV Cache的大小就恢复成跟MHA一样大了，违背了GQA的初衷。

对此，MLA发现，我们可以结合Dot-Attention的具体形式，通过一个简单但不失巧妙的恒等变换来规避这个问题。首先，在训练阶段还是照常进行，此时优化空间不大；然后，在推理阶段，我们利用

$$\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top} = (\mathbf{x}_t \mathbf{W}_q^{(s)}) (\mathbf{c}_i \mathbf{W}_k^{(s)})^\top = \mathbf{x}_t (\mathbf{W}_q^{(s)} \mathbf{W}_k^{(s)\top}) \mathbf{c}_i^\top \quad (6)$$

这意味着推理阶段，我们可以将 $\mathbf{W}_q^{(s)} \mathbf{W}_k^{(s)\top}$ 合并起来作为Q的投影矩阵，那么 $\mathbf{c}_i$ 则取代了原本的 $\mathbf{k}_i$ ，同理，在 $\mathbf{o}_t$ 后面我们还有一个投影矩阵，于是 $\mathbf{v}_i^{(s)} = \mathbf{c}_i \mathbf{W}_v^{(s)}$ 的 $\mathbf{W}_v^{(s)}$ 也可以吸收到后面的投影矩阵中去，于是等效地 $\mathbf{v}_i$ 也可以用 $\mathbf{c}_i$ 代替，也就是说此时KV Cache只需要存下所有的 $\mathbf{c}_i$ 就行，而不至于存下所有的 $\mathbf{k}_i^{(s)}, \mathbf{v}_i^{(s)}$ 。注意到 $\mathbf{c}_i$ 跟 (s) 无关，也就是说所有头共享的，即MLA在推理阶段它可以恒等变换为一个MQA。

再次强调，本文的主题一直都是减少KV Cache，那到目前为止，MLA做到了什么呢？答案是通过不同的投影矩阵来增强了GQA的能力，并且推理时可以保持同样大小的KV Cache。那么反过来，如果我们只需要跟GQA相近的能力，那么是不是就可以再次减少KV Cache了？换言之， d_c 没必要取 $g(d_k + d_v)$ ，而是取更小的值（DeepSeek-V2取了512），从而进一步压缩KV Cache，这就是MLA的核心思想。

（注：这里有一个细节，就是 $\mathbf{W}_q^{(s)} \mathbf{W}_k^{(s)\top}$ 合并成一个矩阵的恒等变换，理论上只有在无限精度下才成立，实际上如果我们使用单精度尤其是BF16的话，经过变换后的精度损失往往还是挺明显的，经过多层累积后可能放大到比较可观的程度，这里可能要根据实际误差看要不要做一些后处理。）

Part 2

一切似乎都很完美，看上去一个又好又省的理想设计就要出炉了。不过别急，当我们再深入思考一下就会发现，到目前为止的MLA有一个难以绕开的缺陷——不兼容RoPE（旋转位置编码）。

刚才我们说了，MLA之所以能保持跟GQA一样大小的KV Cache，其关键一步是“将 $\mathbf{W}_q^{(s)} \mathbf{W}_k^{(s)\top}$ 合并成一个（跟位置无关的）矩阵作为Q的投影矩阵”，但如果加了RoPE的话，这一步就无法实现了。这是因为RoPE是一个跟位置相关的、 $d_k \times d_k$ 的分块对角矩阵 $\mathbf{R}_m$ ，满足 $\mathbf{R}_m \mathbf{R}_n^\top = \mathbf{R}_{m-n}$ ，MLA加入RoPE之后会让 $\mathbf{W}_q^{(s)} \mathbf{W}_k^{(s)\top}$ 之间多插入了一项 $\mathbf{R}_{t-i}$ ：

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \mathcal{R}_i, \quad \mathbf{k}_i^{(s)} = \mathbf{c}_i \mathbf{W}_k^{(s)} \mathcal{R}_i \\ \mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top} &= \left(\mathbf{x}_t \mathbf{W}_q^{(s)} \mathcal{R}_t \right) \left(\mathbf{c}_i \mathbf{W}_k^{(s)} \mathcal{R}_i \right)^\top = \mathbf{x}_t \left(\mathbf{W}_q^{(s)} \mathcal{R}_{t-i} \mathbf{W}_k^{(s)\top} \right) \mathbf{c}_i^\top \end{aligned} \quad (7)$$

这里的 $\mathbf{W}_q^{(s)} \mathcal{R}_{t-i} \mathbf{W}_k^{(s)\top}$ 就无法合并为一个固定的投影矩阵了（跟位置差 $t-i$ 相关），从而MLA的想法无法结合RoPE实现。

前段时间，笔者也很荣幸跟DeepSeek团队讨论过这个问题，但这个问题可以说非常本质，所以当时笔者实际上也没能提出什么有效的建议。最简单的方式是放弃RoPE，换用其他基于Attention Bias的位置编码，如ALIBI，但DeepSeek的实验显示它明显不如RoPE（注意，MLA不是不能加RoPE，而是加了RoPE之后无法用恒等变换技巧来减少KV Cache），笔者也提议过换Sandwich，它不像ALIBI单调衰减到负无穷，估计效果会好些，但感觉是治标不治本。还有一个折中的办法是将 $\mathbf{q}_i$ 的输入也改为 $\mathbf{c}_i$ ，然后RoPE加在 $\mathbf{c}_i$ 之后，即

$$\mathbf{q}_i^{(s)} = \mathbf{c}_i \mathcal{R}_i \mathbf{W}_q^{(s)}, \quad \mathbf{k}_i^{(s)} = \mathbf{c}_i \mathcal{R}_i \mathbf{W}_k^{(s)} \quad (8)$$

这样 $\mathcal{R}_i$ 就可以吸收到 $\mathbf{c}_i$ 中去，但这样就没有 $\mathcal{R}_m \mathcal{R}_n^\top = \mathcal{R}_{m-n}$ 的运算了，此时的RoPE不再是通过绝对位置实现相对位置，而单纯是在Q、K上加绝对位置，让模型自己想办法提炼相对位置信息。

最后发布的MLA，采取了一种混合的方法——每个Attention Head的Q、K新增 d_r 个维度用来添加RoPE，其中K新增的维度每个Head共享：

$$\begin{aligned} \mathbf{o}_t &= [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}] \\ \mathbf{o}_t^{(s)} &= \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})} \\ \mathbf{q}_i^{(s)} &= [\mathbf{x}_i \mathbf{W}_{qc}^{(s)}, \mathbf{x}_i \mathbf{W}_{qr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k+d_r}, \quad \mathbf{W}_{qc}^{(s)} \in \mathbb{R}^{d \times d_k}, \mathbf{W}_{qr}^{(s)} \in \mathbb{R}^{d \times d_r} \\ \mathbf{k}_i^{(s)} &= [\mathbf{c}_i \mathbf{W}_{kc}^{(s)}, \mathbf{x}_i \mathbf{W}_{kr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k+d_r}, \quad \mathbf{W}_{kc}^{(s)} \in \mathbb{R}^{d_c \times d_k}, \mathbf{W}_{kr}^{(s)} \in \mathbb{R}^{d \times d_r} \\ \mathbf{v}_i^{(s)} &= \mathbf{c}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, \quad \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_c \times d_v} \\ \mathbf{c}_i &= \mathbf{x}_i \mathbf{W}_c \in \mathbb{R}^{d_c}, \quad \mathbf{W}_c \in \mathbb{R}^{d \times d_c} \end{aligned} \quad (9)$$

这样一来，没有RoPE的维度就可以重复“Part 1”的操作，在推理时KV Cache只需要存 $\mathbf{c}_i$ ，新增的带RoPE的维度就可以用来补充位置信息，并且由于所有Head共享，所以也就只有在K Cache这里增加了 d_r 个维度，原论文取了 $d_r = d_k/2 = 64$ ，相比原本的 $d_c = 512$ ，增加的幅度不大。

Part 3

最后有一个细节，就是MLA的最终版本，还将Q的输入也改为了低秩投影形式，这与减少KV Cache无关，主要是为了减少训练期间参数量和相应的梯度（原论文说的是激活值，个人表示不大理解）所占的显存：

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})}$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= [\mathbf{c}'_i \mathbf{W}_{qc}^{(s)}, \mathbf{c}'_i \mathbf{W}_{qr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k+d_r}, & \mathbf{W}_{qc}^{(s)} &\in \mathbb{R}^{d'_c \times d_k}, \mathbf{W}_{qr}^{(s)} \in \mathbb{R}^{d'_c \times d_r} \\ \mathbf{k}_i^{(s)} &= [\mathbf{c}_i \mathbf{W}_{kc}^{(s)}, \mathbf{x}_i \mathbf{W}_{kr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_k+d_r}, & \mathbf{W}_{kc}^{(s)} &\in \mathbb{R}^{d_c \times d_k}, \mathbf{W}_{kr}^{(s)} \in \mathbb{R}^{d_c \times d_r} \\ \mathbf{v}_i^{(s)} &= \mathbf{c}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(s)} &\in \mathbb{R}^{d_c \times d_v} \\ \mathbf{c}'_i &= \mathbf{x}_i \mathbf{W}'_c \in \mathbb{R}^{d'_c}, & \mathbf{W}'_c &\in \mathbb{R}^{d \times d'_c} \\ \mathbf{c}_i &= \mathbf{x}_i \mathbf{W}_c \in \mathbb{R}^{d_c}, & \mathbf{W}_c &\in \mathbb{R}^{d \times d_c} \end{aligned} \quad (10)$$

注意 $\mathbf{k}_i^{(s)}$ 中的第二项，带RoPE的部分，其输入还是 $\mathbf{x}_i$ 而不是 $\mathbf{c}_i$ ，这里保持了原论文的设置，不是笔误， d'_c 原论文的取值是1536，跟 $d_c = 512$ 不同。同时，我们把带RoPE的MHA放在下面，方便大家对比：

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)}, \mathbf{o}_t^{(2)}, \dots, \mathbf{o}_t^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{v}_{\leq t}^{(s)}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{v}_i^{(s)}}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})} \quad (11)$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_q^{(s)} \mathcal{R}_i \in \mathbb{R}^{d_k}, & \mathbf{W}_q^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{k}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_k^{(s)} \mathcal{R}_i \in \mathbb{R}^{d_k}, & \mathbf{W}_k^{(s)} &\in \mathbb{R}^{d \times d_k} \\ \mathbf{v}_i^{(s)} &= \mathbf{x}_i \mathbf{W}_v^{(s)} \in \mathbb{R}^{d_v}, & \mathbf{W}_v^{(s)} &\in \mathbb{R}^{d \times d_v} \end{aligned}$$

可以发现，其实在训练阶段，除了多了一步低秩投影以及只在部分维度加RoPE外，MLA与Q、K的Head Size由 d_k 换成 $d_k + d_r$ 的MHA基本无异。

推理阶段的MLA则改为

$$\mathbf{o}_t = [\mathbf{o}_t^{(1)} \mathbf{W}_v^{(1)}, \mathbf{o}_t^{(2)} \mathbf{W}_v^{(2)}, \dots, \mathbf{o}_t^{(h)} \mathbf{W}_v^{(h)}]$$

$$\mathbf{o}_t^{(s)} = \text{Attention}(\mathbf{q}_t^{(s)}, \mathbf{k}_{\leq t}^{(s)}, \mathbf{c}_{\leq t}) \triangleq \frac{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top}) \mathbf{c}_i}{\sum_{i \leq t} \exp(\mathbf{q}_t^{(s)} \mathbf{k}_i^{(s)\top})}$$

$$\begin{aligned} \mathbf{q}_i^{(s)} &= [\mathbf{c}'_i \mathbf{W}_{qc}^{(s)} \mathbf{W}_{kc}^{(s)\top}, \mathbf{c}'_i \mathbf{W}_{qr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_c+d_r} \\ \mathbf{k}_i^{(s)} &= [\mathbf{c}_i, \mathbf{x}_i \mathbf{W}_{kr}^{(s)} \mathcal{R}_i] \in \mathbb{R}^{d_c+d_r} \\ \mathbf{W}_{qc}^{(s)} &\in \mathbb{R}^{d'_c \times d_k}, \mathbf{W}_{kc}^{(s)} \in \mathbb{R}^{d_c \times d_k}, \mathbf{W}_{qr}^{(s)} \in \mathbb{R}^{d'_c \times d_r}, \mathbf{W}_{kr}^{(s)} \in \mathbb{R}^{d_c \times d_r} \\ \mathbf{c}'_i &= \mathbf{x}_i \mathbf{W}'_c \in \mathbb{R}^{d'_c}, & \mathbf{W}'_c &\in \mathbb{R}^{d \times d'_c} \\ \mathbf{c}_i &= \mathbf{x}_i \mathbf{W}_c \in \mathbb{R}^{d_c}, & \mathbf{W}_c &\in \mathbb{R}^{d \times d_c} \end{aligned} \quad (12)$$

此时Q、K的Head Size变成了 $d_c + d_r$ ，V的Head Size 则变成了 d_c ，按照原论文的设置，这是 d_k 、 d_v 的4倍。所以实际上MLA在推理阶段做的这个转换，虽然能有效减少KV Cache，但其推理的计算量是增加的。

那为什么还能提高推理效率呢？这又回到“瓶颈”一节所讨论的问题了，我们可以将LLM的推理分两部分：第一个Token的生成（Prefill）和后续每个Token的生成（Generation），Prefill阶段涉及到对输入所有Token的并行计算，然后把对应的KV Cache存下来，这部分对于计算、带宽和显存都是瓶颈，MLA虽然增大了计算量，但KV Cache的减少也降低了显存和带宽的压力，大家半斤八两；但是Generation阶段由于每步只计算一个Token，实际上它更多的是带宽瓶颈和显存瓶颈，因此MLA的引入理论上能明显提高Generation的速度。

还有一个细节充分体现了这个特性。一般的LLM架构参数满足 $h \times d_k = d$ ，即`num_heads * head_size = hidden_size`，但DeepSeek-V2不一样，它 $d_k = 128, d = 5120$ ，但 $h = 128$ ，是一般设置的3倍！这是因为MLA的KV Cache大小跟 h 无关，增大 h 只会增加计算量和提升模型能力，但不会增加KV Cache，所以不会带来速度瓶颈。

小结

本文简单概述了多头注意力的演变历程，特别是从MHA向MQA、GQA，最终到MLA的变化理念，最后详细展开了对MLA的介绍。在本文中，MLA被视为GQA的一般化，它用投影矩阵的方式替代了GQA的分割、重复，并引入了一个恒等变换技巧来可以进一步压缩KV Cache，同时采用了一种混合方法来兼容RoPE。总的来说，MLA称得上是一种非常实用的注意力变体。

转载到请包括本文地址：<https://spaces.ac.cn/archives/10091>
更详细的转载事宜请参考：《科学空间FAQ》

如果您需要引用本文，请参考：

苏剑林. (May. 13, 2024). 《缓存与效果的极限拉扯：从MHA、MQA、GQA到MLA》[Blog post]. Retrieved from <https://spaces.ac.cn/archives/10091>

```
@online{kexuefm-10091,
  title={缓存与效果的极限拉扯：从MHA、MQA、GQA到MLA},
  author={苏剑林},
  year={2024},
  month={May},
  url={\url{https://spaces.ac.cn/archives/10091}},
}
```